
Práctica 5

Arquitecturas

Paralelas

Universidad de Córdoba
Escuela Politécnica Superior

-

Grado de Ingeniería Informática

-

Arquitecturas Paralelas

★ Juan Manuel Ramírez Hinojosa (i22rahij)



Ejercicio Práctica V

Con el programa de multiplicación de matrices de la práctica I, implementar versiones paralelas utilizando OpenMp. Las matrices deberán tener un tamaño de 4000x4000 y se deberá:

1. Realizar una implementación paralela (con número de hilos por defecto) con vector de iteraciones (i, j, k) paralelizando: a) el bucle más externo, b) el bucle intermedio y c) el bucle más interno. Con los resultados de tiempo obtenidos desarrollar unas conclusiones.
 - a. Con la opción del mejor tiempo, realizar pruebas contemplando distinto número de hilos (2, 4, 8, 16, 32, 64) y discutir resultados en base a speed-up y eficiencia.
2. Realizar una implementación paralela (con número de hilos por defecto) con vector de iteraciones (j, k, i) paralelizando: a) el bucle más externo, b) el bucle intermedio y c) el bucle más interno. Con los resultados de tiempo obtenidos desarrollar unas conclusiones.
 - a. Con la opción del mejor tiempo, realizar pruebas contemplando distinto número de hilos (2, 4, 8, 16, 32, 64) y discutir resultados en base a speed-up y eficiencia.
3. Realizar una implementación paralela (con número de hilos por defecto) con vector de iteraciones (i, k, j) paralelizando: a) el bucle más externo, b) el bucle intermedio y c) el bucle más interno. Con los resultados de tiempo obtenidos desarrollar unas conclusiones.
 - a. Con la opción del mejor tiempo, realizar pruebas contemplando distinto número de hilos (2, 4, 8, 16, 32, 64) y discutir resultados en base a speed-up y eficiencia.
4. Realizar la ejecución secuencial del programa con el mismo número y vectores de iteraciones anteriores y comparar resultados de dicha ejecución secuencial con las mejores opciones de paralelización de los 3 puntos anteriores. Discutir los resultados y realizar unas conclusiones sobre los mismos.
5. ¿Aplicarías mejoras en alguna de las implementaciones paralelas anteriores para obtener mejores tiempos? ¿Qué mejora y dónde?

Notas:

1.- Para el apartado 5. Hay que tener en cuenta que, en la comparación entre experimentos, el número de iteraciones debe ser el mismo para que las pruebas sean comparables.



Apartado 1

Hemos modificado el código implementado en la práctica 1 y hemos hecho los cambios pertinentes para cumplir con lo establecido en el enunciado, realizando varias funciones para paralelizar cada bucle, la cual vamos cambiando previo a compilación. Antes de comprobar la manera de iterar más rápida hemos decidido emplear 4 hilos como número por defecto. El código resultante es el siguiente:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4000 // Dimensión de la matriz
#define NUM_THREADS 4 // Número de hilos

int A[N][N], B[N][N], C[N][N];
int i, j, k;

void multiply_matrices_i() {
    #pragma omp parallel for private(j, k)
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void multiply_matrices_j() {
    for (i = 0; i < N; i++) {
        #pragma omp parallel for private(k)
        for (j = 0; j < N; j++) {
            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

void multiply_matrices_k() {
    for (i = 0; i < N; i++) {
```



```

        for (j = 0; j < N; j++) {
            #pragma omp parallel for
            for (k = 0; k < N; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}

int main() {
    int i, j, k;

    // Inicialización de matrices A y B con valores aleatorios
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            A[i][j] = rand() % 10;
            B[i][j] = rand() % 10;
            C[i][j] = 0;
        }
    }

    omp_set_num_threads(NUM_THREADS);
    double start = omp_get_wtime();

    // Multiplicación de matrices paralelizada por filas
    multiply_matrices_i();

    double end = omp_get_wtime();

    // Validar un valor para evitar optimización
    printf("C[100][100] = %d\n", C[100][100]);

    double elapsed_ms = (end - start) * 1000.0;
    printf("Tiempo transcurrido con %d hilos = %.2f ms\n", NUM_THREADS,
elapsed_ms);
    printf("Tiempo promedio por hilo (estimado) = %.2f ms\n",
elapsed_ms / NUM_THREADS);
}

```



Tras realizar pruebas paralelizando cada uno de los bucles hemos obtenido los siguientes tiempos:

```
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 4 hilos = 80520.25 ms
Tiempo promedio por hilo (estimado) = 20130.06 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80162
Tiempo transcurrido con 4 hilos = 90898.94 ms
Tiempo promedio por hilo (estimado) = 22724.73 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 117069
Tiempo transcurrido con 4 hilos = 602326.74 ms
Tiempo promedio por hilo (estimado) = 150581.68 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$
```

La ejecución de los bloques se ha ejecutado en el orden de más externo a más interno y, como podemos comprobar, el más rápido es aquel en el que paralelizamos el bucle *i*, es decir, el más externo, por lo que para el resto de este apartado realizaremos las pruebas paralelizando el bucle *i*.

Paralelizar el bucle externo *i* en la multiplicación de matrices es más eficiente porque cada hilo trabaja con filas completas de la matriz *C*, lo que evita conflictos de escritura y mejora el uso de la memoria caché. Al distribuir bloques de trabajo independientes, se minimiza la necesidad de sincronización entre hilos, lo que permite una ejecución más rápida y escalable.

En cambio, paralelizar los bucles *j* o *k* puede generar accesos concurrentes a las mismas posiciones de la matriz resultado, lo que introduce conflictos de escritura y pérdida de rendimiento. Además, estos bucles suelen tener peores patrones de acceso a memoria, lo que reduce la eficiencia del uso de la caché y puede incluso invalidar los resultados si no se controlan correctamente los accesos concurrentes.

Tras esto, podemos pasar al análisis del speed-up con varios números de hilos paralelizando el bucle *i*. Para ello realizamos el mismo proceso ejecutado en anteriores prácticas y obtenemos la siguiente tabla de datos y su correspondiente gráfica.

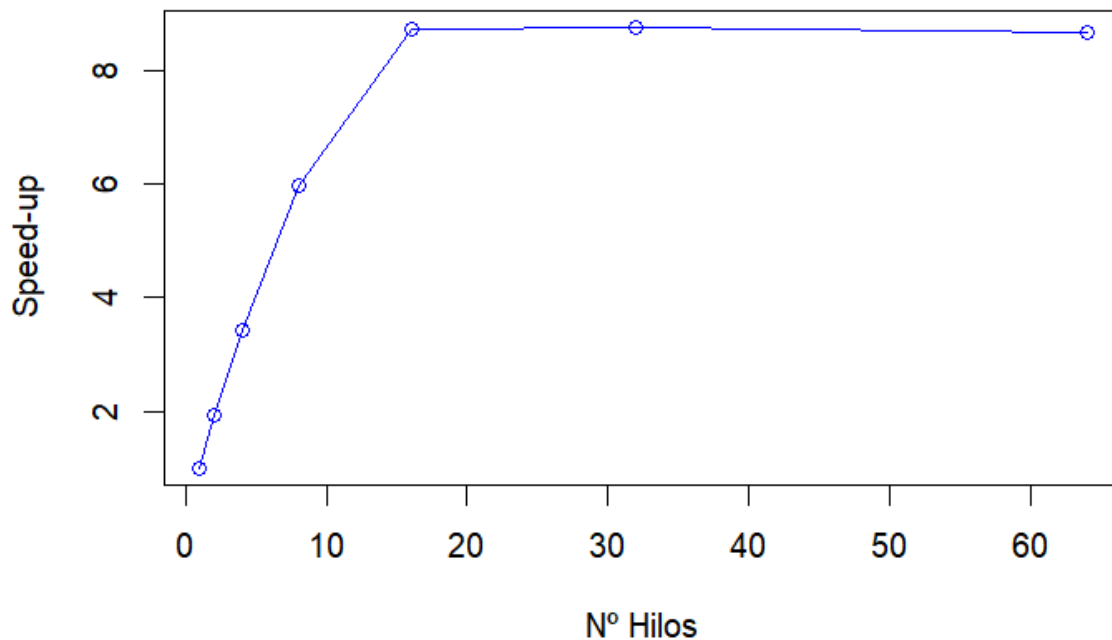


	I-J-K						
Nº Hilos	1	2	4	8	16	32	64
Tiempo Medio	273038	141206	79772	45564	31209	31179	31441
Valor más alto	274883	147440	86199	46767	31438	31336	31497
Valor más bajo	271145	134045	76082	44772	31027	31127	31354
Variación	3738	13395	10117	1995	411	209	143
Speed-Up	1.00	1.93	3.42	5.99	8.75	8.76	8.68

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos

Curva de rendimiento I-J-K



Apartado 2

Para estos siguientes apartados empleamos versiones ligeramente modificadas del código anterior y realizamos el mismo proceso de ejecución. Como podemos ver el bucle más externo, en este caso j, vuelve a ser el más rápido por lo que realizaremos las pruebas de este apartado con el mismo.

```
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_jki.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 4 hilos = 133572.83 ms
Tiempo promedio por hilo (estimado) = 33393.21 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_jki.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 78701
Tiempo transcurrido con 4 hilos = 162563.65 ms
Tiempo promedio por hilo (estimado) = 40640.91 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_jki.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 4 hilos = 147794.86 ms
Tiempo promedio por hilo (estimado) = 36948.72 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ |
```

En este caso, podemos concluir que paralelizar el bucle j (el más externo) sea más rápido se debe a que esta estrategia divide de forma más equilibrada el trabajo entre hilos desde el principio, asignando grandes bloques de columnas completas a cada hilo. Esto reduce la sobrecarga de creación, sincronización y administración de hilos, y permite un mayor aprovechamiento del paralelismo a gran escala. Cada hilo ejecuta muchas iteraciones de forma continua, lo que favorece una mejor utilización de la caché y menos cambios de contexto.

Tras esto, podemos pasar al análisis del speed-up con varios números de hilos paralelizando el bucle j. Para ello realizamos el mismo proceso ejecutado en anteriores prácticas y obtenemos la siguiente tabla de datos y su correspondiente gráfica.

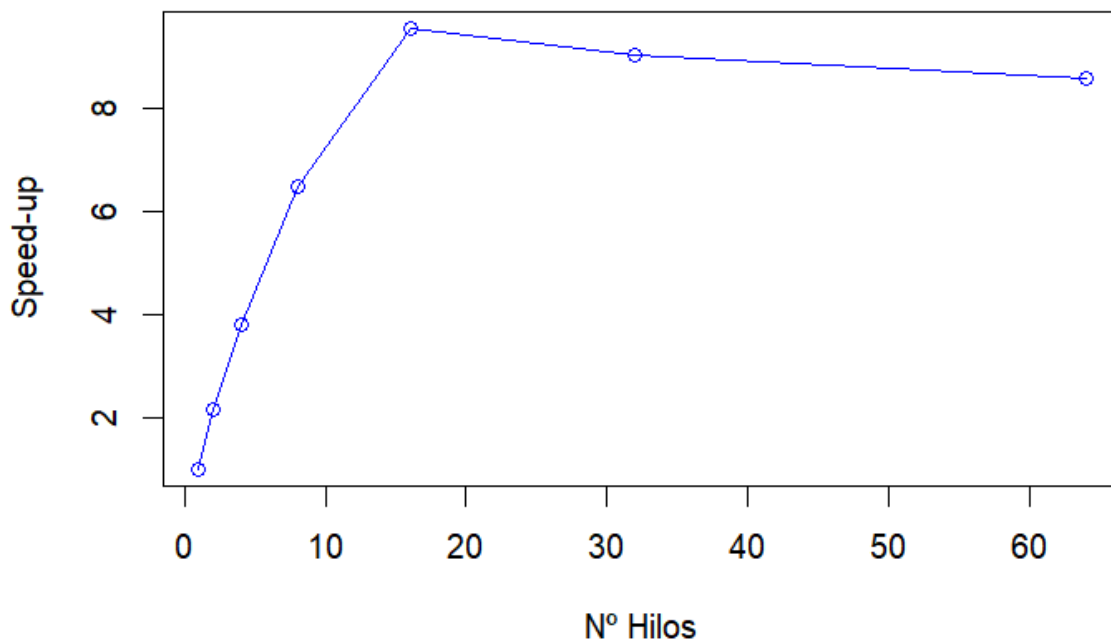


	J-K-I						
Nº Hilos	1	2	4	8	16	32	64
Tiempo Medio	515954	240871	135934	79321	53999	56985	59968
Valor más alto	524913	247911	158703	80919	56178	59836	60504
Valor más bajo	509282	233789	122745	77942	53328	55150	59061
Variación	15631	14122	35958	2977	2850	4686	1443
Speed-Up	1.00	2.14	3.80	6.50	9.55	9.05	8.60

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos

Curva de rendimiento J-K-I



Apartado 3

Una vez realizado el proceso de ejecución comprobamos que el bucle más externo, en este caso i, vuelve a ser el más rápido por las mismas razones explicadas en anteriores apartados.

```

juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_ikj.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 4 hilos = 62031.69 ms
Tiempo promedio por hilo (estimado) = 15507.92 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_ikj.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 79877
Tiempo transcurrido con 4 hilos = 82612.05 ms
Tiempo promedio por hilo (estimado) = 20653.01 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_ikj.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 4 hilos = 98825.09 ms
Tiempo promedio por hilo (estimado) = 24706.27 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ |

```

Tras esto, podemos pasar al análisis del speed-up con varios números de hilos paralelizando el bucle j. Para ello realizamos el mismo proceso ejecutado en anteriores prácticas y obtenemos la siguiente tabla de datos y su correspondiente gráfica.

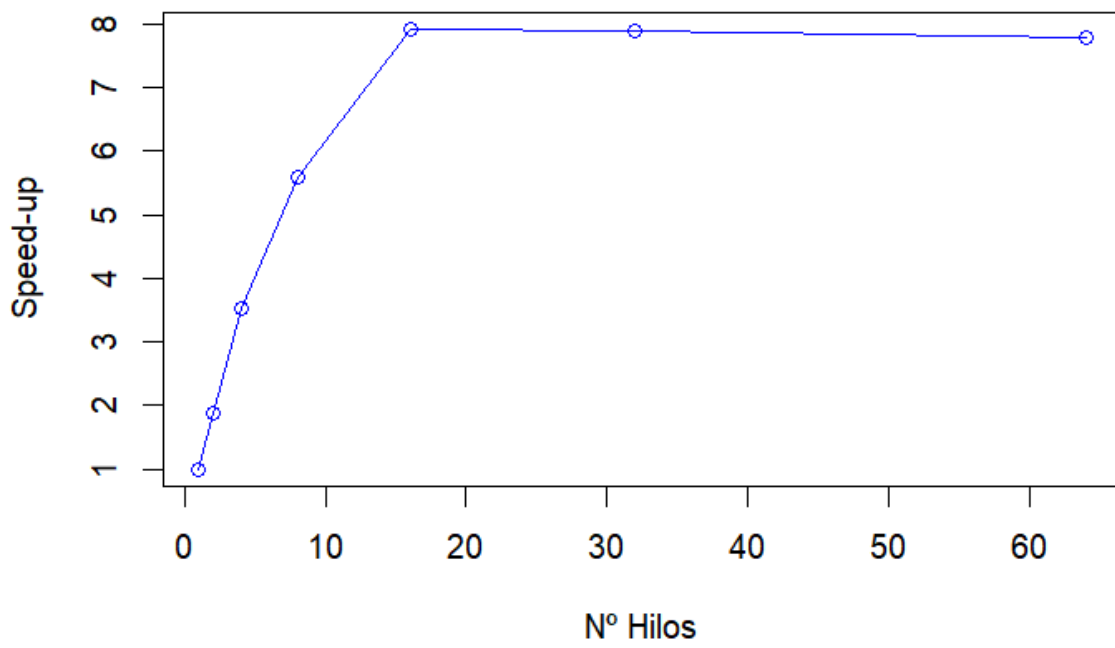
	I-K-J						
Nº Hilos	1	2	4	8	16	32	64
Tiempo Medio	187688	99945	53233	33575	23739	23793	24057
Valor más alto	188132	108441	54083	34615	23790	23842	24678
Valor más bajo	187361	95468	52012	32400	23684	23723	23803
Variación	771	12973	2072	2215	106	119	875
Speed-Up	1.00	1.88	3.53	5.59	7.91	7.89	7.80

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos



Curva de rendimiento I-K-J



Apartado 4

Al analizar los tiempos de ejecución para un solo hilo, se observan los siguientes promedios:

- **I-J-K:** 273,038 ms
- **J-K-I:** 515,954 ms
- **I-K-J:** 187,688 ms

Esto indica que, en ejecución secuencial, el orden de bucles I-K-J es el más eficiente, seguido por I-J-K, y el más lento es J-K-I. Esta diferencia se explica por el acceso a la memoria: I-K-J favorece un mejor uso de la caché gracias a un acceso más secuencial a las filas de las matrices, mientras que J-K-I provoca más saltos de memoria y peores tasas de acierto en la caché.

Si comparamos ahora con los mejores tiempos paralelos (el menor tiempo medio obtenido para cada orden), tenemos:

- **I-J-K (16 hilos):** 31,209 ms
- **J-K-I (16 hilos):** 53,999 ms
- **I-K-J (16 hilos):** 23,739 ms

Nuevamente, I-K-J es el más rápido, tanto en secuencial como en paralelo. Esto confirma que el orden de bucles I-K-J es el más eficiente globalmente, pues mantiene buena coherencia de caché y permite una paralelización efectiva. El orden J-K-I, incluso con paralelización, sigue siendo el más lento debido a la mala localización de memoria, lo cual limita las ganancias obtenidas con múltiples hilos.

Con todo esto podemos concluir que el orden de bucles tiene un impacto significativo en el rendimiento, incluso más allá del paralelismo.

La combinación I-K-J + paralelización con 16 hilos ofrece el mejor rendimiento global, alcanzando un speed-up de 7.91x respecto a su versión secuencial por las razones previamente explicadas.

A partir de 16 hilos, las mejoras se estabilizan o incluso decrecen ligeramente, lo que indica una saturación de recursos pues el procesador empleado para las pruebas solo es capaz de abastecer a un máximo de 16 hilos.



Apartado 5

Sí, podemos aplicar una serie de mejoras para así reducir el tiempo de ejecución. Para comenzar, se añadió la cláusula `schedule(static)` en la directiva `#pragma omp parallel for`, lo que hace que OpenMP divida el trabajo de manera equitativa y fija entre los hilos (por bloques de iteraciones).

Por otra parte, hemos usado una variable temporal `temp` para optimizar el acceso a `A[i][k]`. Sin esta variable, el acceso a `A[i][k]` se realizaría repetidamente dentro del bucle `j`, generando múltiples lecturas innecesarias desde memoria.

Hemos decidido aplicar estas mejoras a la ejecución con 16 hilos del bucle I-K-J, pues es el que presenta mejor tiempo.

El código resultante tras estas modificaciones es el mostrado a continuación.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4000          // Dimensión de la matriz
#define NUM_THREADS 16 // Número de hilos

int A[N][N], B[N][N], C[N][N];
int i, j, k; // Variables de control de bucle

void multiply_matrices_ikj() {
    #pragma omp parallel for schedule(static) private( i, k, j)
    for (int i = 0; i < N; i++) {
        for (int k = 0; k < N; k++) {
            int temp = A[i][k];
            for (int j = 0; j < N; j++) {
                C[i][j] += temp * B[k][j];
            }
        }
    }
}

int main() {
    // Inicialización de matrices A y B con valores aleatorios
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i][j] = rand() % 10;
            B[i][j] = rand() % 10;
            C[i][j] = 0;
        }
    }
}
```



```

}

omp_set_num_threads(NUM_THREADS);
double start = omp_get_wtime();

// Multiplicación de matrices optimizada
multiply_matrices_ikj();

double end = omp_get_wtime();

// Validar un valor para evitar optimización
printf("C[100][100] = %d\n", C[100][100]);

double elapsed_ms = (end - start) * 1000.0;
printf("Tiempo transcurrido con %d hilos = %.2f ms\n", NUM_THREADS,
elapsed_ms);
printf("Tiempo promedio por hilo (estimado) = %.2f ms\n",
elapsed_ms / NUM_THREADS);

return 0;
}

```

Una vez aplicadas estas mejoras hemos realizado el proceso normal de ejecución y hemos obtenido los siguientes tiempos:

```

juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ gcc practica5_ikj.c -fopenmp
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 16 hilos = 18472.75 ms
Tiempo promedio por hilo (estimado) = 1154.55 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 16 hilos = 18390.61 ms
Tiempo promedio por hilo (estimado) = 1149.41 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 16 hilos = 18404.24 ms
Tiempo promedio por hilo (estimado) = 1150.26 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 16 hilos = 18878.75 ms
Tiempo promedio por hilo (estimado) = 1179.92 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ ./a.out
C[100][100] = 80111
Tiempo transcurrido con 16 hilos = 18430.43 ms
Tiempo promedio por hilo (estimado) = 1151.90 ms
juanmrhinojosa04@DESKTOP-JFN8RE3:/mnt/c/Users/Juan Manuel/Desktop/uni/AP/p5$ |

```



	Sin Mejoras	Mejorado
Nº Hilos	16	16
Tiempo Medio	23739	18514
Valor más alto	23790	18878
Valor más bajo	23684	18390
Variación	106	488
Speed-Up	7.91	10.14
Mejora	1.00	1.28

* Tiempo medido en milisegundos

* $\text{Speed-up}(p) = T_1/T_p$; T_p = tiempo de ejecución con p hilos

Como podemos observar, las mejoras aplicadas han dado lugar a un speed-up de 10.14 respecto al 7.91 sin las mejoras (respecto al programa secuencial sin mejorar) y, a su vez, una mejora del 28% con respecto al tiempo previo a los cambios.

